MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

ML for Symptom Classification Based on Cough Audio Signals

Divya Pothavajhyula, Pooja Vinukonda, Mary Czelusniak

Agenda

- Project Statement
- Project Challenges
- Embedded Systems Component
- Machine Learning Component
- Putting Them Together
- Implementation and Evaluations
- Video Demo
- Limitations and Future Work

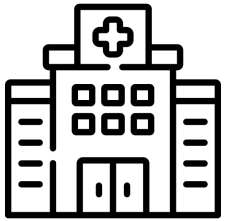
Project Statement

- Respiratory conditions (e.g., cold, flu) are common & widespread
- Smartphones are ‘always’ available sensing platforms
- Existing solutions rely on cloud processing

Need: On-device, non-invasive, privacy-preserving cough analysis

Goal: Real-time cough pattern classification (non-diagnostic)

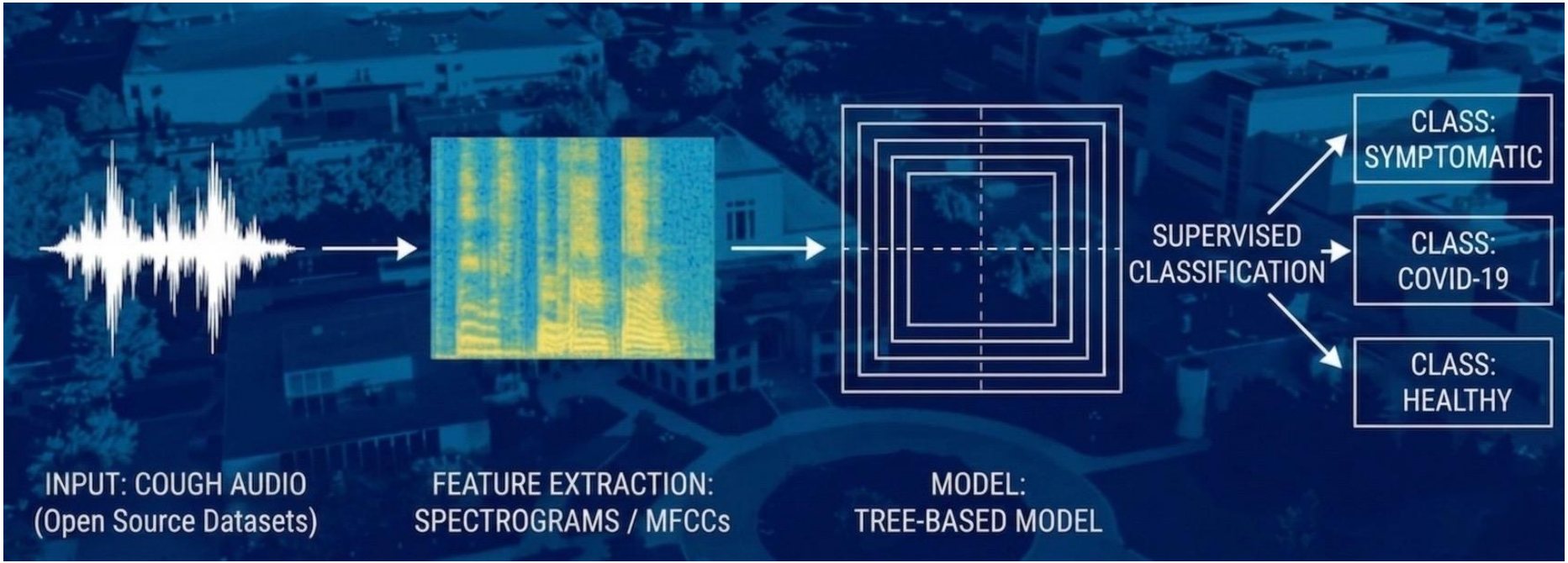
- Target classes: healthy vs. symptomatic vs. COVID-19



Project Challenges

- Data Quality & Preprocessing
- Cough Segmentation
- Feature Representation
- Safe Testing of “Unhealthy” Cases
- On-Device Constraints

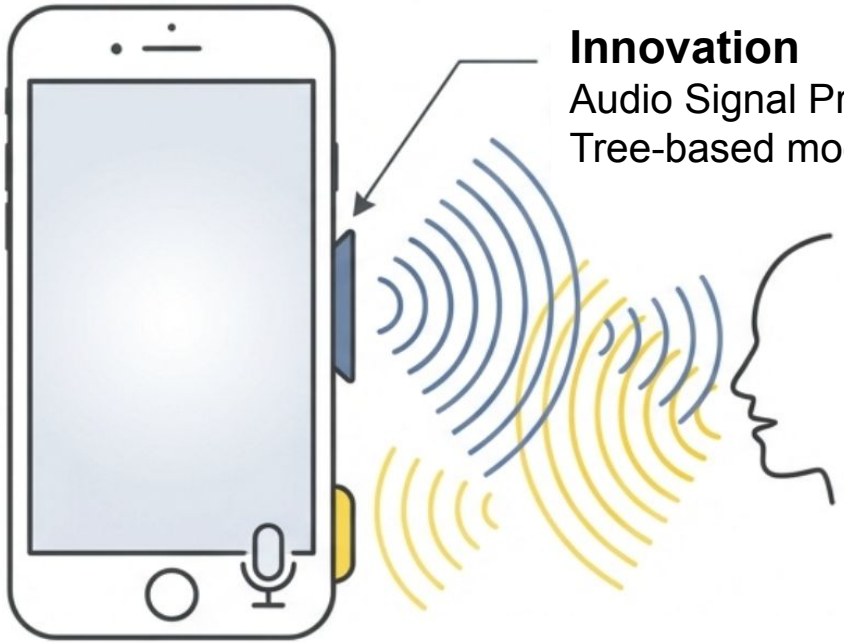
Healthy vs. Symptomatic


INPUT: COUGH AUDIO
(Open Source Datasets)
FEATURE EXTRACTION:
SPECTROGRAMS / MFCCs
MODEL:
TREE-BASED MODEL
SUPERVISED
CLASSIFICATION
CLASS:
SYMPTOMATIC
CLASS:
COVID-19
CLASS:
HEALTHY

Computational Sensors & Device Setup

Platform
Smartphone (or laptop)
System constraints: energy, compute, memory, latency

Innovation
Audio Signal Processing
Tree-based models



Primary Sensor
Built-in microphone
Real-time audio capture

Fully On-Device Pipeline: Real-time audio capture → feature extraction → inference

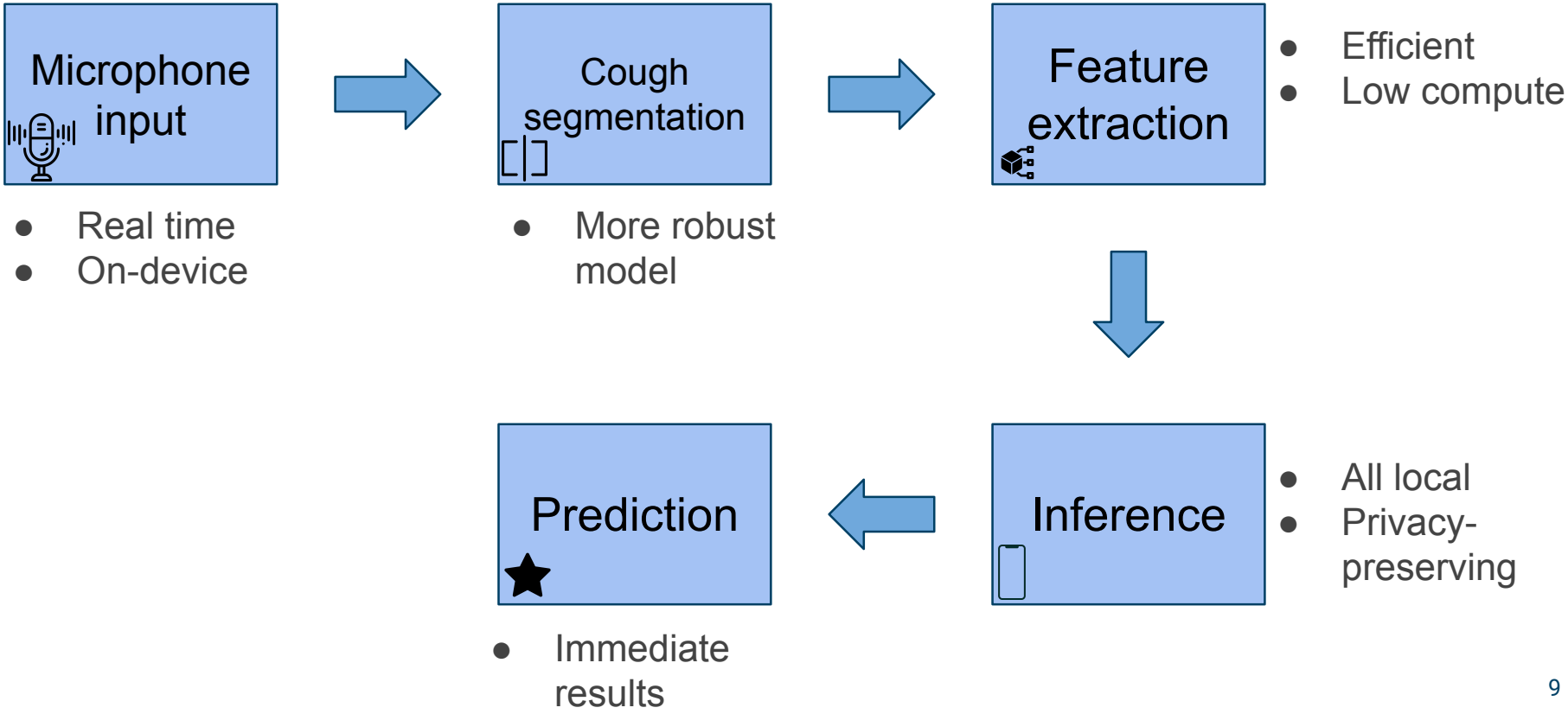
Embedded Systems Component

- **On-Device Audio Sensing:** Utilizing the device microphone to capture real-time cough audio signals
- **Local Real-Time Inference Pipeline:** Recording short audio segments and processing them immediately for classification and running a trained XGBoost model directly on-device without cloud support
- **Signal Processing on Device:** Converting audio into Mel spectrogram representations suitable for model input

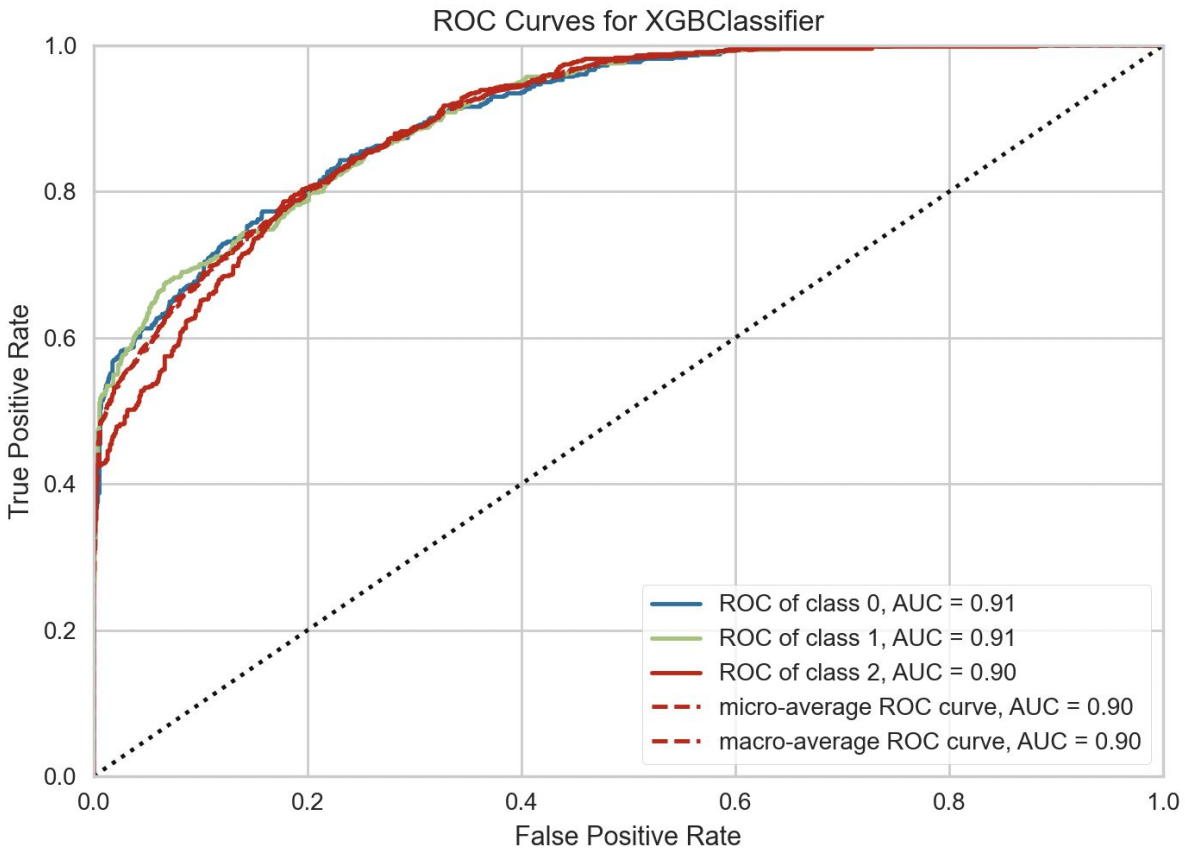
ML Component

- **Classification Task:** categorize coughs into *healthy*, *symptomatic,* or *covid-19*
- **Dataset:** COUGHVID from Kaggle
  - 25K+ recordings → filtered to include only ‘good quality recordings’
  - Used a balanced subset
  - Isolated individual coughs
  - Extracted features to capture temporal / frequency characteristics
- **Model:** XGBoost via PyCaret library
- **Optimization:** Normalization + feature transformations → balance classification accuracy with computational efficiency

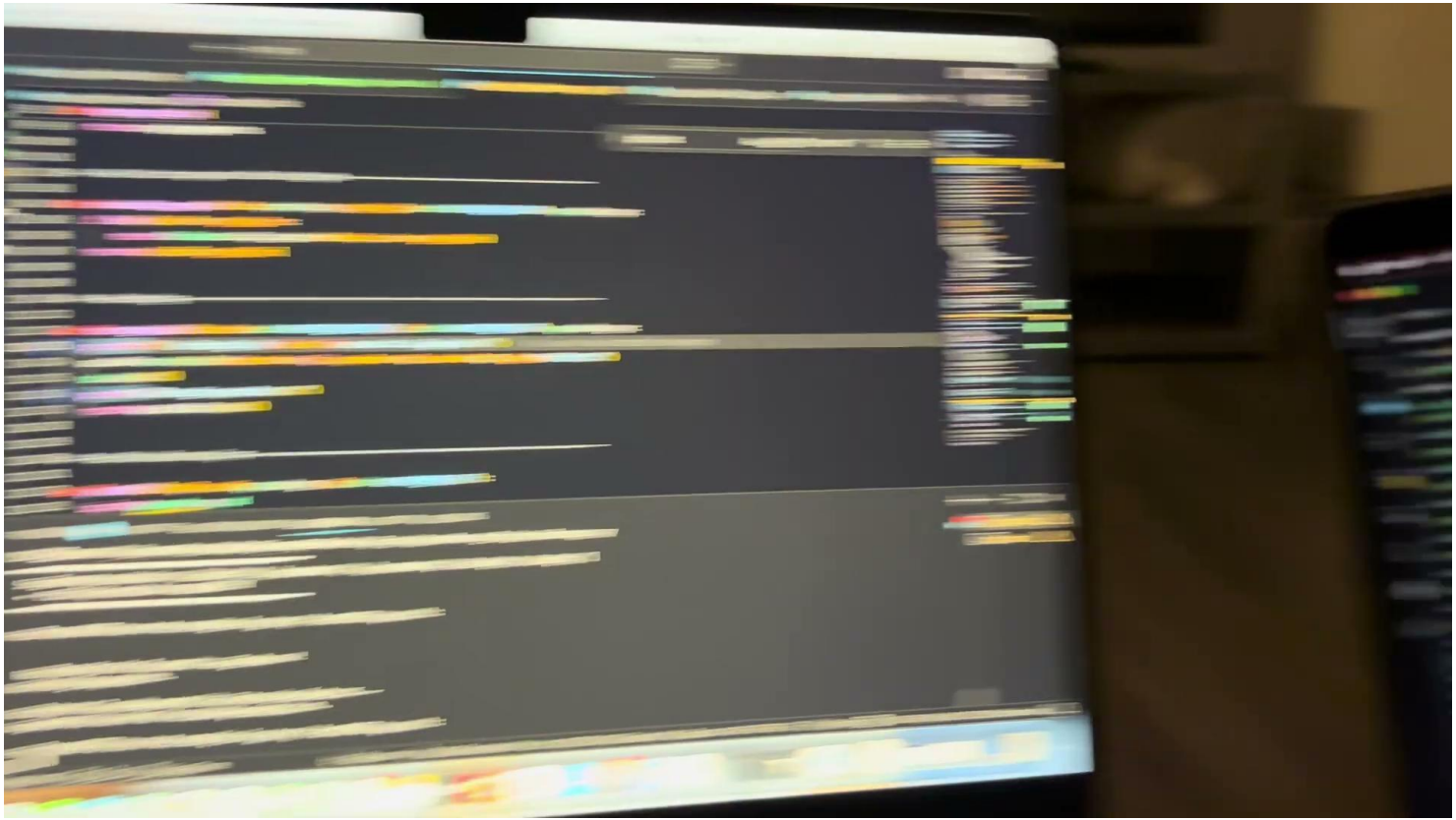
Putting Them Together: End-to-End Pipeline



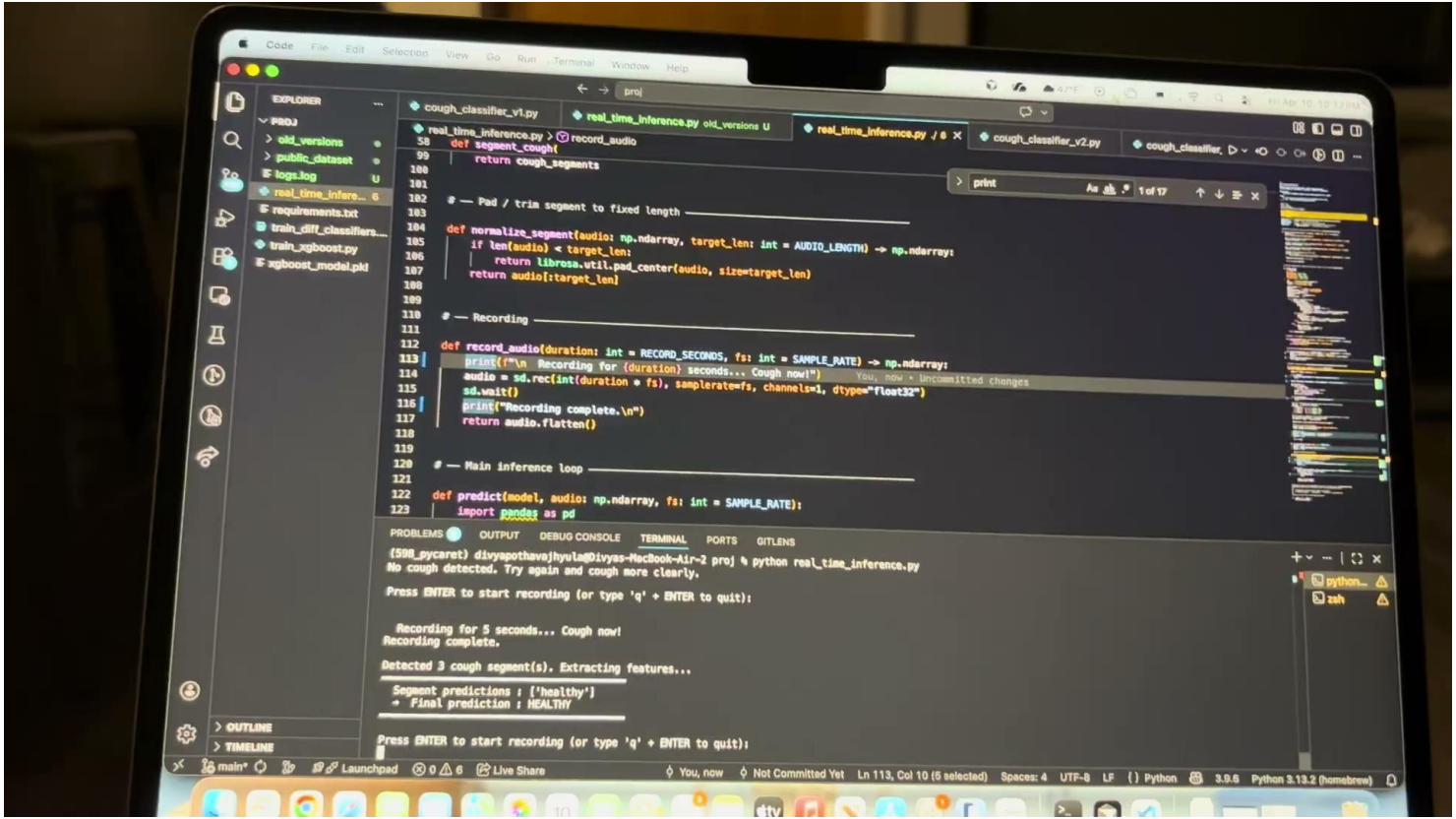
Implementations/Evaluations



Video Demo: COVID-19



Video Demo: Healthy



Limitations & Future Work

- **Environmental Noise:** Difficulty distinguishing cough signals from background noise (e.g., conversations, TV) in real-world settings
  → **Future Work:** Incorporate noise-robust features and filtering techniques
- **Real-Time Latency vs. Accuracy:** Trade-off between feature extraction complexity and fast on-device inference
  → **Future Work:** Optimize feature pipeline and explore model compression/pruning
- **Limited Real-World "Unhealthy" Data:** Challenges in safely collecting live unhealthy cough samples; reliance on curated datasets or playback
  → **Future Work:** Expand dataset with more diverse, real-world recordings
- **Generalization Across Devices:** Variability in microphone quality and recording conditions may impact performance
  → **Future Work:** Test across devices and apply normalization/adaptation techniques

Discussion & Feedback

ML Component

Supervised classification of cough audio segments into respiratory condition categories (healthy, symptomatic, COVID-19) using structured audio features.

- **Features:** Raw audio signals are preprocessed by segmenting individual cough events using signal power thresholds. Each segment is transformed into feature vectors including MFCCs, Mel-spectrograms, chroma features, and spectral contrast to capture temporal and frequency characteristics.
- **Model:** Using tree-based ensemble methods (Random Forest / XGBoost) via PyCaret for robust classification on tabular feature representations, enabling strong performance without heavy neural network computation.
- **Dataset:** Utilizing the COUGHVID dataset with physician-validated labels. Data is filtered for high-quality cough recordings and balanced across classes to improve training stability.
- **Optimization:** Applying normalization and feature transformations, along with model comparison and tuning, to balance classification accuracy with computational efficiency for potential real-time, on-device deployment.

Putting Them Together

- **End-to-End Pipeline:** Microphone input → cough segmentation → feature extraction → XGBoost classification → output prediction
- **Seamless ML + Sensing Integration:** Combining real-time audio capture with on-device signal processing and machine learning inference
- **Lightweight Processing Flow:** Using efficient feature extraction (MFCCs, spectral features) to enable fast model input generation
- **Real-Time Decision Making:** Generating immediate cough classification results directly on-device without external computation
- **Privacy-Preserving Architecture:** Entire pipeline runs locally, ensuring user data never leaves the device
- **Deployment-Oriented Design:** System designed with constraints in mind (latency, compute, memory) for practical mobile/embedded use

Motivation and Background

Respiratory monitoring on mobile/embedded devices is non-invasive, accessible, and preserve user privacy
Goal: non-diagnostic screening, identify cough characteristics and symptom indicators (not a medical diagnosis)

Target classes (examples):

- Dry vs. wet cough
- Healthy vs. symptomatic cough patterns

Why on-device?

- Keeps raw audio private (no cloud upload)
- Enables real-time feedback for users

Expected impact: show that meaningful respiratory-related signal patterns can be inferred from cough audio on resource-constrained devices

Motivation and Background

- **Problem:** Respiratory health monitoring is often inaccessible or invasive; there is a growing need for non-diagnostic, privacy-preserving screening tools.
- **Core Goal:** Develop an on-device ML system to classify cough characteristics (e.g., dry vs. wet) using smartphone microphones.
- **Privacy-First Design:** All processing occurs locally on-device to safeguard user data; no raw audio is stored or uploaded to the cloud.

Computational Sensors & Device Setup

- Platform: **smartphone and/or laptop**
  - Uses **built-in microphone** as the primary sensor
- Pipeline runs **fully locally**
  - Real-time audio capture → feature extraction → inference
  - **No raw audio stored** to protect privacy
- System constraints to evaluate:
  - **Latency / responsiveness**
  - **Compute + memory footprint**
  - **Energy usage** (especially on phones)
- Stretch goal (multimodal sensing):
  - Use **ultrasound / Doppler shift** to estimate cough-to-mic distance
  - Provide user guidance (e.g., “move closer”) to improve model performance